6

Functions and an Introduction to Recursion

OBJECTIVES

- To construct programs modularly from functions.
- To create functions with multiple parameters.
- The mechanisms for passing information between functions and returning results.
- Understand function call stack and activation records

OBJECTIVES

- To use random number generation to implement game-playing applications.
- How the visibility of identifiers is limited to specific regions of programs.
- To write and use recursive functions, i.e., functions that call themselves.

6.1 Introduction

- **Divide and conquer technique**
  - **Construct a large program from small, simple pieces (e.g., components).**
- **Functions**
  - **Facilitate the design, implementation, operation and maintenance of large programs.**

6.2 Program Components in C++

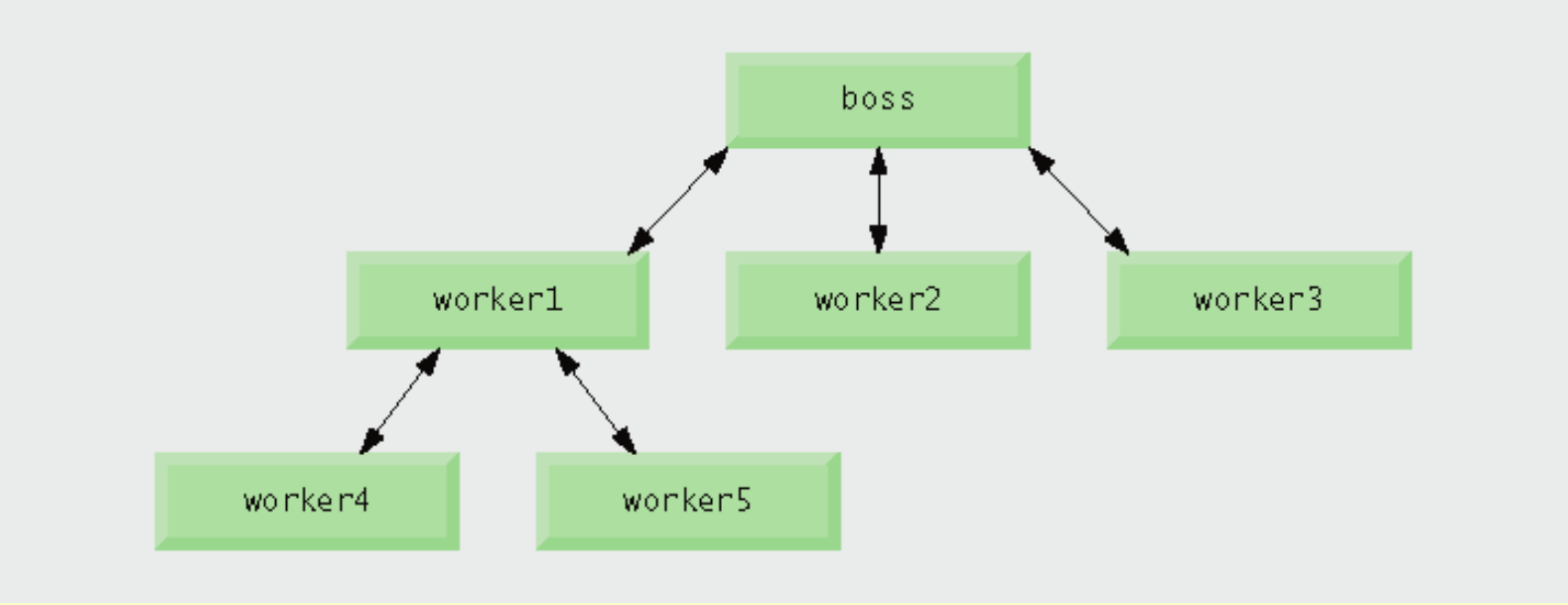
- **C++ Standard Library**
  - **Rich collection of functions for performing common operations, such as:**
    - **Mathematical calculations**
    - **String manipulations**
    - **Character manipulations**
    - **Input/Output**
    - **Error checking**

6.2 Program Components in C++ (Cont.)

- **Functions**
  - **Are called methods or procedures in other languages.**
  - **Statements in function bodies are written only once.**
    - **Are reused from perhaps several locations in a program.**
    - **Are hidden from other functions.**
    - **Avoid repeating code.**

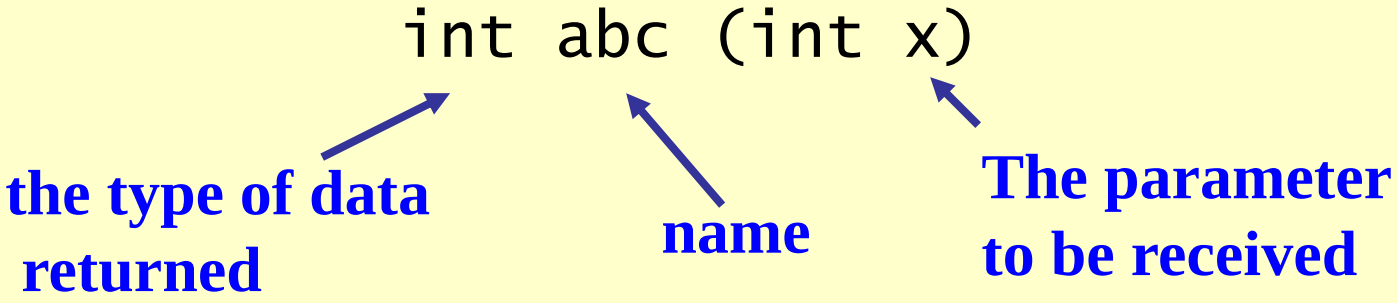
6.2 Program Components in C++ (cont.)

- **A function is invoked by a function call.**
  - **Called function either returns a result or simply returns to the caller.**
  - **Function calls form hierarchical relationships.**
  - **It is similar to a member function except this time no object is created.**



6.3 Math Library Functions

- **Global functions**
  - **Do not belong to a particular class.**
  - **Have function prototypes placed in header files.**
    - **Function prototypes include the name of the function, the type of data returned by the function, and the parameters to be received by the function.**



6.3 Math Library Functions

- Example: `sqrt` in `<cmath>` header file
  - All functions in `<cmath>` are global functions.

Function	Description	Example
`ceil( x )`	rounds *x* to the smallest integer not less than *x*	`ceil( 9.2 )` is 10.0 `ceil( -9.8 )` is -9.0
`cos( x )`	trigonometric cosine of *x* (*x* in radians)	`cos( 0.0 )` is 1.0
`exp( x )`	exponential function e^x	`exp( 1.0 )` is 2.71828 `exp( 2.0 )` is 7.38906
`fabs( x )`	absolute value of *x*	`fabs( 5.1 )` is 5.1 `fabs( 0.0 )` is 0.0 `fabs( -8.76 )` is 8.76
`floor( x )`	rounds *x* to the largest integer not greater than *x*	`floor( 9.2 )` is 9.0 `floor( -9.8 )` is -10.0
`fmod( x, y )`	remainder of *x*/*y* as a floating-point number	`fmod( 2.6, 1.2 )` is 0.2
`log( x )`	natural logarithm of *x* (base *e*)	`log( 2.718282 )` is 1.0 `log( 7.389056 )` is 2.0
`log10( x )`	logarithm of *x* (base 10)	`log10( 10.0 )` is 1.0 `log10( 100.0 )` is 2.0
`pow( x, y )`	*x* raised to power *y* (x^y)	`pow( 2, 7 )` is 128 `pow( 9, .5 )` is 3
`sin( x )`	trigonometric sine of *x* (*x* in radians)	`sin( 0.0 )` is 0
`sqrt( x )`	square root of *x* (where *x* is a nonnegative value)	`sqrt( 9.0 )` is 3.0
`tan( x )`	trigonometric tangent of *x* (*x* in radians)	`tan( 0.0 )` is 0

6.4 Function Definitions with Multiple Parameters

- **Multiple parameters**
  - **Specified in both the function prototype and the function header as a comma-separated list of parameters.**
  - **Each argument must be consistent with the type of the corresponding parameter.**
  - **Parameters are also called formal parameter.**

```
1  // Fig. 6.3: GradeBook.h
2  // Definition of class GradeBook that finds the maximum of three grades.
3  // Member functions are defined in GradeBook.cpp
4  #include <string> // program uses C++ standard string class
5  using std::string;
6
7  // GradeBook class definition
8  class GradeBook
9  {
10 public:
11    GradeBook( string ); // constructor initializes course name
12    void setCourseName( string ); // function to set the course name
13    string getCourseName(); // function to retrieve the course name
14    void displayMessage(); // display a welcome message
15    void inputGrades(); // input three grades from user
16    void displayGradeReport(); // display a report based on the grades
17    int maximum( int, int, int ); // determine max of 3 values
18 private:
19    string courseName; // course name for this GradeBook
20    int maximumGrade; // maximum of three grades
21 }; // end class GradeBook
```

Outline

GradeBook.h

(1 of 1)

```
1  // Fig. 6.4: GradeBook.cpp
2  // Member-function definitions for class GradeBook that
3  // determines the maximum of three grades.
4  #include <iostream>
5  using std::cout;
6  using std::cin;
7  using std::endl;
8
9  #include "GradeBook.h" // include definition of class GradeBook
10
11 // constructor initializes courseName with string supplied as argument;
12 // initializes studentMaximum to 0
13 GradeBook::GradeBook( string name )
14 {
15    setCourseName( name ); // validate and store courseName
16    maximumGrade = 0; // this value will be replaced by the maximum grade
17 } // end GradeBook constructor
18
19 // function to set the course name; limits name to 25 or fewer characters
20 void GradeBook::setCourseName( string name )
21 {
22    if ( name.length() <= 25 ) // if name has 25 or fewer characters
23       courseName = name; // store the course name in the object
24    else // if name is longer than 25 characters
25    { // set courseName to first 25 characters of parameter name
26       courseName = name.substr( 0, 25 ); // select first 25 characters
27       cout << "Name \"" << name << "\" exceeds maximum length (25).\n"
28          << "Limiting courseName to first 25 characters.\n" << endl;
29    } // end if...else
30 } // end function setCourseName
```

Outline

GradeBook.cpp

(1 of 3)

```
31
32 // function to retrieve the course name
33 string GradeBook::getCourseName()
34 {
35    return courseName;
36 } // end function getCourseName
37
38 // display a welcome message to the GradeBook user
39 void GradeBook::displayMessage()
40 {
41    // this statement calls getCourseName to get the
42    // name of the course this GradeBook represents
43    cout << "Welcome to the grade book for\n" << getCourseName() << "!\n"
44       << endl;
45 } // end function displayMessage
46
47 // input three grades from user; determine maximum
48 void GradeBook::inputGrades()
49 {
50    int grade1; // first grade entered by user
51    int grade2; // second grade entered by user
52    int grade3; // third grade entered by user
53
54    cout << "Enter three integer grades: ";
55    cin >> grade1 >> grade2 >> grade3;
56
57    // store maximum in member studentMaximum
58    maximumGrade = maximum( grade1, grade2, grade3 );
59 } // end function inputGrades
```

Outline

GradeBook.cpp

(2 of 3)

```
60
61 // returns the maximum of its three integer parameters
62 int GradeBook::maximum( int x, int y, int z )
63 {
64    int maximumValue = x; // assume x is the largest to start
65
66    // determine whether y is greater than maximumValue
67    if ( y > maximumValue )
68       maximumValue = y; // make y the new maximumValue
69
70    // determine whether z is greater than maximumValue
71    if ( z > maximumValue )
72       maximumValue = z; // make z the new maximumValue
73
74    return maximumValue;
75 } // end function maximum
76
77 // display a report based on the grades entered by user
78 void GradeBook::displayGradeReport()
79 {
80    // output maximum of grades entered
81    cout << "Maximum of grades entered: " << maximumGrade << endl;
82 } // end function displayGradeReport
```

Outline

GradeBook.cpp

(3 of 3)

```
1  // Fig. 6.5: fig06_05.cpp
2  // Create GradeBook object, input grades and display grade report.
3  #include "GradeBook.h" // include definition of class GradeBook
4
5  int main()
6  {
7     // create GradeBook object
8     GradeBook myGradeBook( "CS101 C++ Programming" );
9
10    myGradeBook.displayMessage(); // display welcome message
11    myGradeBook.inputGrades(); // read grades from user
12    myGradeBook.displayGradeReport(); // display report based on grades
13    return 0; // indicate successful termination
14 } // end main
```

```
Welcome to the grade book for
CS101 C++ Programming!

Enter three integer grades: 86 67 75
Maximum of grades entered: 86
```

```
Welcome to the grade book for
CS101 C++ Programming!

Enter three integer grades: 67 86 75
Maximum of grades entered: 86
```

```
Welcome to the grade book for
CS101 C++ Programming!

Enter three integer grades: 67 75 86
Maximum of grades entered: 86
```

Outline

fig06_05.cpp

(1 of 1)

6.4 Function Definitions with Multiple Parameters (Cont.)

- **Three ways to return control to the calling statement:**
  - **If the function does not return a result (i.e., `void`):**
    - **Program flow reaches the function-ending right brace or**
    - **Program executes the statement `return;`**
  - **If the function does return a result:**
    - **Program executes the statement `return` *expression*`;`**
      - ***expression* is evaluated and its value is returned to the caller.**

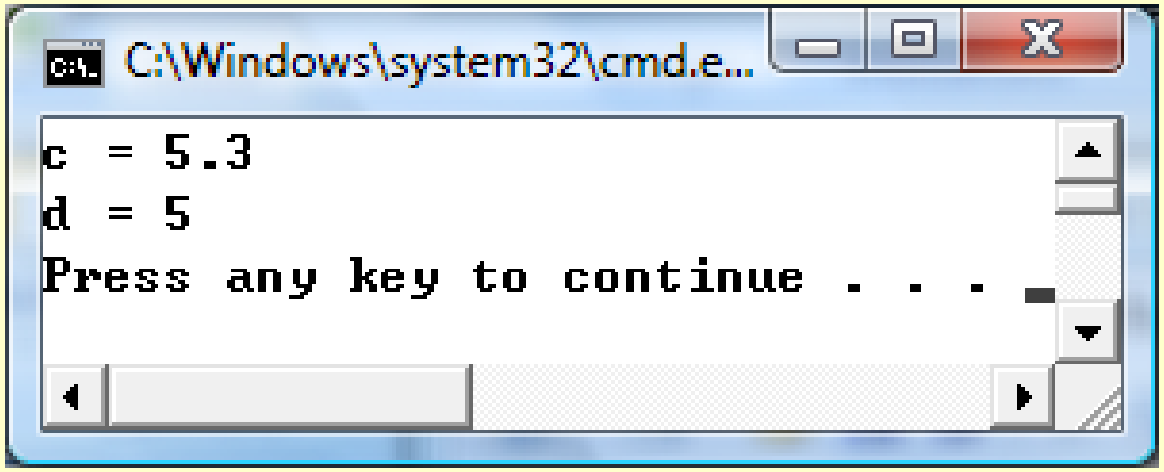
6.5 Function Prototypes and Argument Coercion (Cont.)

- **Argument Coercion**
  - **Forcing arguments to the appropriate types specified by the corresponding parameters.**
- **C++ Promotion Rules**
  - **Indicate how to convert between types without losing data.**
  - **Apply to expressions containing values of two or more data types.**

6.5 Function Prototypes and Argument Coercion (Cont.)

– **Example 1:**

```
int a = 3;
double b = 2.3;
double c = a + b;
int d = a + b;
```



6.5 Function Prototypes and Argument Coercion (Cont.)

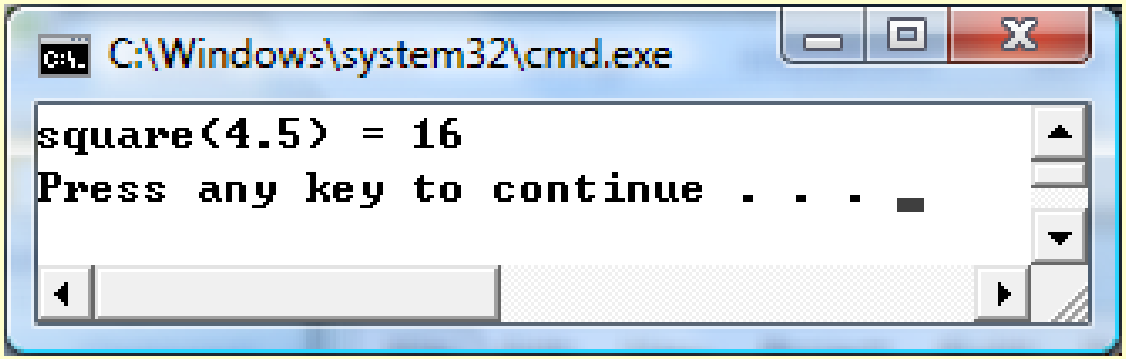
- **Promoting hierarchy for fundamental data type from "highest type" to "lowest type".**

Data types	
`long double`	
`double`	
`float`	
`unsigned long int`	(synonymous with `unsigned long`)
`long int`	(synonymous with `long`)
`unsigned int`	(synonymous with `unsigned`)
`int`	
`unsigned short int`	(synonymous with `unsigned short`)
`short int`	(synonymous with `short`)
`unsigned char`	
`char`	
`bool`	

6.5 Function Prototypes and Argument Coercion (Cont.)

– **Example 2:**

```
int square( int a )
{
  return a * a;
}
...
cout << “square(4.5) = “ << square(4.5) << endl;
```



6.7 Case Study: Random Number Generation

- **C++ Standard Library function `rand`**
  - `i = rand();`
  - **Randomly generates an unsigned integer between `0` and `RAND_MAX`**
    - **RAND_MAX is a symbolic constant defined in header file `<cstdlib>`.**
  - **Usually `RAND_MAX` = 32767**
    - **Maximum positive value for two-byte (16-bit) integers, i.e., $2^{16} / 2 - 1 = 32767$ (-32767 to 32767).**
  - **Function prototype for the `rand` function is in `<cstdlib>`.**

6.7 Case Study: Random Number Generation (Cont.)

- **Scaling and shifting random numbers**
  - **To obtain random numbers in a desired range, use a statement like**

```
number = shiftingValue + rand() % scalingFactor;
```

  - **The integer generated by the above statement will be randomly between *shiftingValue* and *shiftingValue* + (*scalingFactor* – 1).**

6.7 Case Study: Random Number Generation (Cont.)

- **Example 1: *shiftingValue* = 1, *scalingFactor* = 6**

```
1 + rand() % 6;
```

  - **Produces numbers in the range `1` to `6`.**

- **Example 2: *shiftingValue* = 10, *scalingFactor* = 21**

```
10 + rand() % 21;
```

  - **Produces numbers in the range `10` to `30`.**

Class Work

- **Produces numbers in the range 12 to 24.**

Answer:

- **Randomly generate a number from the following even integers: 2, 4, 6, 8, and 10.**

Answer:

```
1  // Fig. 6.8: fig06_08.cpp
2  // Shifted and scaled random integers.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  #include <iomanip>
8  using std::setw;
9
10 #include <cstdlib> // contains function prototype for rand
11 using std::rand;
12
13 int main()
14 {
15    // loop 20 times
16    for ( int counter = 1; counter <= 20; counter++ )
17    {
18       // pick random number from 1 to 6 and output it
19       cout << setw( 10 ) << ( 1 + rand() % 6 );
```

Outline

fig06_08.cpp

(1 of 2)

```
20
21       // if counter is divisible by 5, start a new line of output
22       if ( counter % 5 == 0 )
23          cout << endl;
24    } // end for
25
26    return 0; // indicates successful termination
27 } // end main
```

```
          6          6          5          5          6
          5          1          1          5          3
          6          6          2          4          2
          6          2          3          4          1
```

Outline

fig06_08.cpp

(2 of 2)

Outline

fig06_09.cpp

(1 of 3)

```
1  // Fig. 6.9: fig06_09.cpp
2  // Roll a six-sided die 6,000,000 times.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  #include <iomanip>
8  using std::setw;
9
10 #include <cstdlib> // contains function prototype for rand
11 using std::rand;
12
13 int main()
14 {
15    int frequency1 = 0; // count of 1s rolled
16    int frequency2 = 0; // count of 2s rolled
17    int frequency3 = 0; // count of 3s rolled
18    int frequency4 = 0; // count of 4s rolled
19    int frequency5 = 0; // count of 5s rolled
20    int frequency6 = 0; // count of 6s rolled
21
22    int face; // stores most recently rolled value
23
24    // summarize results of 6,000,000 rolls of a die
25    for ( int roll = 1; roll <= 6000000; roll++ )
26    {
27       face = 1 + rand() % 6; // random number from 1 to 6
```

```
28
29       // determine roll value 1-6 and increment appropriate counter
30       switch ( face )
31       {
32          case 1:
33             ++frequency1; // increment the 1s counter
34             break;
35          case 2:
36             ++frequency2; // increment the 2s counter
37             break;
38          case 3:
39             ++frequency3; // increment the 3s counter
40             break;
41          case 4:
42             ++frequency4; // increment the 4s counter
43             break;
44          case 5:
45             ++frequency5; // increment the 5s counter
46             break;
47          case 6:
48             ++frequency6; // increment the 6s counter
49             break;
50          default: // invalid value
51             cout << "Program should never get here!";
52       } // end switch
53    } // end for
```

Outline

fig06_09.cpp

(2 of 3)

```
54
55    cout << "Face" << setw( 13 ) << "Frequency" << endl; // output headers
56    cout << "    1" << setw( 13 ) << frequency1
57       << "\n    2" << setw( 13 ) << frequency2
58       << "\n    3" << setw( 13 ) << frequency3
59       << "\n    4" << setw( 13 ) << frequency4
60       << "\n    5" << setw( 13 ) << frequency5
61       << "\n    6" << setw( 13 ) << frequency6 << endl;
62    return 0; // indicates successful termination
63 } // end main
```

```
Face      Frequency
   1         999702
   2        1000823
   3         999378
   4         998898
   5        1000777
   6        1000422
```

Outline

fig06_09.cpp

(3 of 3)

6.7 Case Study: Random Number Generation (Cont.)

- **Function `rand`**
  - **Generates pseudorandom numbers.**
  - **The same sequence of numbers repeats itself each time the program executes.**

$X_0 = 100$

$X_1 = 29$

$X_2 = 43$

$X_3 = 3433$

$$X_{n+1} = f(X_n)$$

where *f* is a pseudo random number generator

6.7 Case Study: Random Number Generation (Cont.)

- **Randomizing**
  - **Condition a program to produce a different sequence of random numbers for each execution .**
- **C++ Standard Library function `srand`**
  - **Takes an unsigned integer argument.**
  - **Seeds the `rand` function to produce a different sequence of random numbers.**
  - **The seed is X_0.**

```
1  // Fig. 6.10: fig06_10.cpp
2  // Randomizing die-rolling program.
3  #include <iostream>
4  using std::cout;
5  using std::cin;
6  using std::endl;
7
8  #include <iomanip>
9  using std::setw;
10
11 #include <cstdlib> // contains prototypes for functions srand and rand
12 using std::rand;
13 using std::srand;
14
15 int main()
16 {
17    unsigned seed; // stores the seed entered by the user
18
19    cout << "Enter seed: ";
20    cin >> seed;
21    srand( seed ); // seed random number generator
22
```

Outline

fig06_10.cpp

(1 of 2)

```
23    // loop 10 times
24    for ( int counter = 1; counter <= 10; counter++ )
25    {
26       // pick random number from 1 to 6 and output it
27       cout << setw( 10 ) << ( 1 + rand() % 6 );
28
29       // if counter is divisible by 5, start a new line of output
30       if ( counter % 5 == 0 )
31          cout << endl;
32    } // end for
33
34    return 0; // indicates successful termination
35 } // end main
```

```
Enter seed: 67
         6         1         4         6         2
         1         6         1         6         4
```

```
Enter seed: 432
         4         6         3         1         6
         3         1         5         4         2
```

```
Enter seed: 67
         6         1         4         6         2
         1         6         1         6         4
```

Outline

fig06_10.cpp

(2 of 2)

6.7 Case Study: Random Number Generation (Cont.)

- **To randomize without having to enter a seed each time**
  - `srand( time( 0 ) );`
    - **This causes the computer to read its clock to obtain the seed value.**
  - **Function `time` (with the argument `0`)**
    - **Returns the current time as the number of seconds since January 1, 1970 at midnight Greenwich Mean Time (GMT).**
    - **Function prototype for `time` is in `<ctime>`.**

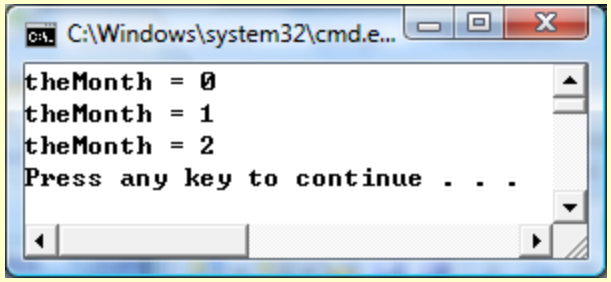
6.8 Case Study: Game of Chance and Introducing enum

- **Enumeration**
  - **A set of integer constants represented by identifiers**
    - **The values of enumeration constants start at `0`, unless specified otherwise, and increment by `1`.**

6.8 Case Study: Game of Chance and Introducing enum

– **Example 1:**

```
enum Month { JAN, FEB, MAR };
Month theMonth;
theMonth = JAN;
cout << "theMonth = " << theMonth << endl;
theMonth = FEB;
cout << "theMonth = " << theMonth << endl;
theMonth = MAR;
cout << "theMonth = " << theMonth << endl;
```

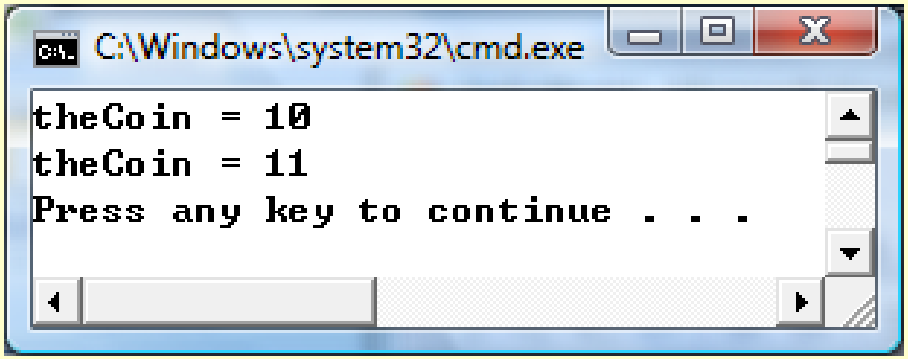


6.8 Case Study: Game of Chance and Introducing enum

– **Example 2:**

```
enum Coin { HEAD = 10, TAIL };
Coin theCoin;

theCoin = HEAD;
cout << "theCoin = " << theCoin << endl;
theCoin = TAIL;
cout << "theCoin = " << theCoin << endl;
```



Outline

fig06_11.cpp

(1 of 4)

```
1  // Fig. 6.11: fig06_11.cpp
2  // Craps simulation.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  #include <cstdlib> // contains prototypes for functions srand and rand
8  using std::rand;
9  using std::srand;
10
11 #include <ctime> // contains prototype for function time
12 using std::time;
13
14 int rollDice(); // rolls dice, calculates amd displays sum
15
16 int main()
17 {
18    // enumeration with constants that represent the game status
19    enum Status { CONTINUE, WON, LOST }; // all caps in constants
20
21    int myPoint; // point if no win or loss on first roll
22    Status gameStatus; // can contain CONTINUE, WON or LOST
23
24    // randomize random number generator using current time
25    srand( time( 0 ) );
26
27    int sumOfDice = rollDice(); // first roll of the dice
```

```
28
29    // determine game status and point (if needed) based on first roll
30    switch ( sumOfDice )
31    {
32       case 7: // win with 7 on first roll
33       case 11: // win with 11 on first roll
34          gameStatus = WON;
35          break;
36       case 2: // lose with 2 on first roll
37       case 3: // lose with 3 on first roll
38       case 12: // lose with 12 on first roll
39          gameStatus = LOST;
40          break;
41       default: // did not win or lose, so remember point
42          gameStatus = CONTINUE; // game is not over
43          myPoint = sumOfDice; // remember the point
44          cout << "Point is " << myPoint << endl;
45          break; // optional at end of switch
46    } // end switch
47
48    // while game is not complete
49    while ( gameStatus == CONTINUE ) // not WON or LOST
50    {
51       sumOfDice = rollDice(); // roll dice again
52
```

Outline

fig06_11.cpp

(2 of 4)

```
53       // determine game status
54       if ( sumOfDice == myPoint ) // win by making point
55          gameStatus = WON;
56       else
57          if ( sumOfDice == 7 ) // lose by rolling 7 before point
58             gameStatus = LOST;
59    } // end while
60
61    // display won or lost message
62    if ( gameStatus == WON )
63       cout << "Player wins" << endl;
64    else
65       cout << "Player loses" << endl;
66
67    return 0; // indicates successful termination
68 } // end main
69
70 // roll dice, calculate sum and display results
71 int rollDice()
72 {
73    // pick random die values
74    int die1 = 1 + rand() % 6; // first die roll
75    int die2 = 1 + rand() % 6; // second die roll
76
77    int sum = die1 + die2; // compute sum of die values
```

Outline

fig06_11.cpp

(3 of 4)

```
78
79    // display results of this roll
80    cout << "Player rolled " << die1 << " + " << die2
81       << " = " << sum << endl;
82    return sum; // end function rollDice
83 } // end function rollDice
```

```
Player rolled 2 + 5 = 7
Player wins
```

```
Player rolled 6 + 6 = 12
Player loses
```

```
Player rolled 3 + 3 = 6
Point is 6
Player rolled 5 + 3 = 8
Player rolled 4 + 5 = 9
Player rolled 2 + 1 = 3
Player rolled 1 + 5 = 6
Player wins
```

```
Player rolled 1 + 3 = 4
Point is 4
Player rolled 4 + 6 = 10
Player rolled 2 + 4 = 6
Player rolled 6 + 4 = 10
Player rolled 2 + 3 = 5
Player rolled 2 + 4 = 6
Player rolled 1 + 1 = 2
Player rolled 4 + 4 = 8
Player rolled 4 + 3 = 7
Player loses
```

Outline

fig06_11.cpp

(4 of 4)

Class Work

- **A set is defined as below:**

```
enum Colors {Red, Green=10, Blue, Yellow=20, White};
```

- **What is the value of each element?**

Answer:

6.9 Storage Classes

- **Global variables**
  - **Declares outside any class or function definition.**
  - **Retain their values throughout the execution of the program.**
  - **`static` is used when global variables are declared to be accessed by any functions in the same file.**
  - **`extern` is used when global variables are declared to be accessed by any functions in different files (i.e., more than one file).**

6.9 Storage Classes (Cont.)

- **Local variables**
  - **Variables declared in a function definition's body**
- **Local variables declared with keyword `static`**
  - **Are known only in the function in which they are declared.**
  - **Retain their values when the function returns to its caller.**
    - **Next time the function is called, the `static` local variables contain the values they had when the function last completed.**
  - **If numeric variables of the static storage class are not explicitly initialized by the programmer, they are initialized to zero.**

```cpp
// Fig. 6.12: fig06_12.cpp
// A scoping example.
#include <iostream>
using std::cout;
using std::endl;

void useLocal( void ); // function prototype
void useStaticLocal( void ); // function prototype
void useGlobal( void ); // function prototype

int x = 1; // global variable

int main()
{
   int x = 5; // local variable to main

   cout << "local x in main's outer scope is " << x << endl;

   { // start new scope
      int x = 7; // hides x in outer scope

      cout << "local x in main's inner scope is " << x << endl;
   } // end new scope

   cout << "local x in main's outer scope is " << x << endl;
```

Outline

fig06_12.cpp

(1 of 4)

```
26
27    useLocal(); // useLocal has local x
28    useStaticLocal(); // useStaticLocal has static local x
29    useGlobal(); // useGlobal uses global x
30    useLocal(); // useLocal reinitializes its local x
31    useStaticLocal(); // static local x retains its prior value
32    useGlobal(); // global x also retains its value
33
34    cout << "\nlocal x in main is " << x << endl;
35    return 0; // indicates successful termination
36 } // end main
37
38 // useLocal reinitializes local variable x during each call
39 void useLocal( void )
40 {
41    int x = 25; // initialized each time useLocal is called
42
43    cout << "\nlocal x is " << x << " on entering useLocal" << endl;
44    x++;
45    cout << "local x is " << x << " on exiting useLocal" << endl;
46 } // end function useLocal
```

Outline

fig06_12.cpp

(2 of 4)

Outline

```
47
48 // useStaticLocal initializes static local variable x only the
49 // first time the function is called; value of x is saved
50 // between calls to this function
51 void useStaticLocal( void )
52 {
53    static int x = 50; // initialized first time useStaticLocal is called
54
55    cout << "\nlocal static x is " << x << " on entering useStaticLocal"
56       << endl;
57    x++;
58    cout << "local static x is " << x << " on exiting useStaticLocal"
59       << endl;
60 } // end function useStaticLocal
61
62 // useGlobal modifies global variable x during each call
63 void useGlobal( void )
64 {
65    cout << "\nglobal x is " << x << " on entering useGlobal" << endl;
66    x *= 10;
67    cout << "global x is " << x << " on exiting useGlobal" << endl;
68 } // end function useGlobal
```

```
local x in main's outer scope is 5
local x in main's inner scope is 7
local x in main's outer scope is 5

local x is 25 on entering useLocal
local x is 26 on exiting useLocal

local static x is 50 on entering useStaticLocal
local static x is 51 on exiting useStaticLocal

global x is 1 on entering useGlobal
global x is 10 on exiting useGlobal

local x is 25 on entering useLocal
local x is 26 on exiting useLocal

local static x is 51 on entering useStaticLocal
local static x is 52 on exiting useStaticLocal

global x is 10 on entering useGlobal
global x is 100 on exiting useGlobal

local x in main is 5
```

Outline

fig06_12.cpp

(4 of 4)

6.11 Function Call Stack and Activation Records

- **Function Call Stack**
  - **Sometimes is called the program execution stack.**
  - **Each time a function is called, a stack frame (also known as an activation record) is pushed onto the stack.**
  - **An activation record includes return address and variables declared in the function.**
  - **When a function returns, the record is popped and the control transfers to the caller's address.**

6.11 Function Call Stack and Activation Records (Cont.)

- **Stack overflow**
  - **Error that occurs when more function calls occur than can have their activation records stored on the function call stack (due to memory limitations).**

```
1  // Fig. 6.13: fig06_13.cpp
2  // square function used to demonstrate the function
3  // call stack and activation records.
4  #include <iostream>
5  using std::cin;
6  using std::cout;
7  using std::endl;
8
9  int square( int ); // prototype for function square
10
11 int main()
12 {
13    int a = 10; // value to square (local automatic variable in main)
14
15    cout << a << " squared: " << square( a ) << endl; // display a squared
16    return 0; // indicate successful termination
17 } // end main
18
19 // returns the square of an integer
20 int square( int x ) // x is a local variable
21 {
22    return x * x; // calculate square and return result
23 } // end function square
```

```
10 squared: 100
```

Outline

fig06_13.cpp

(1 of 1)

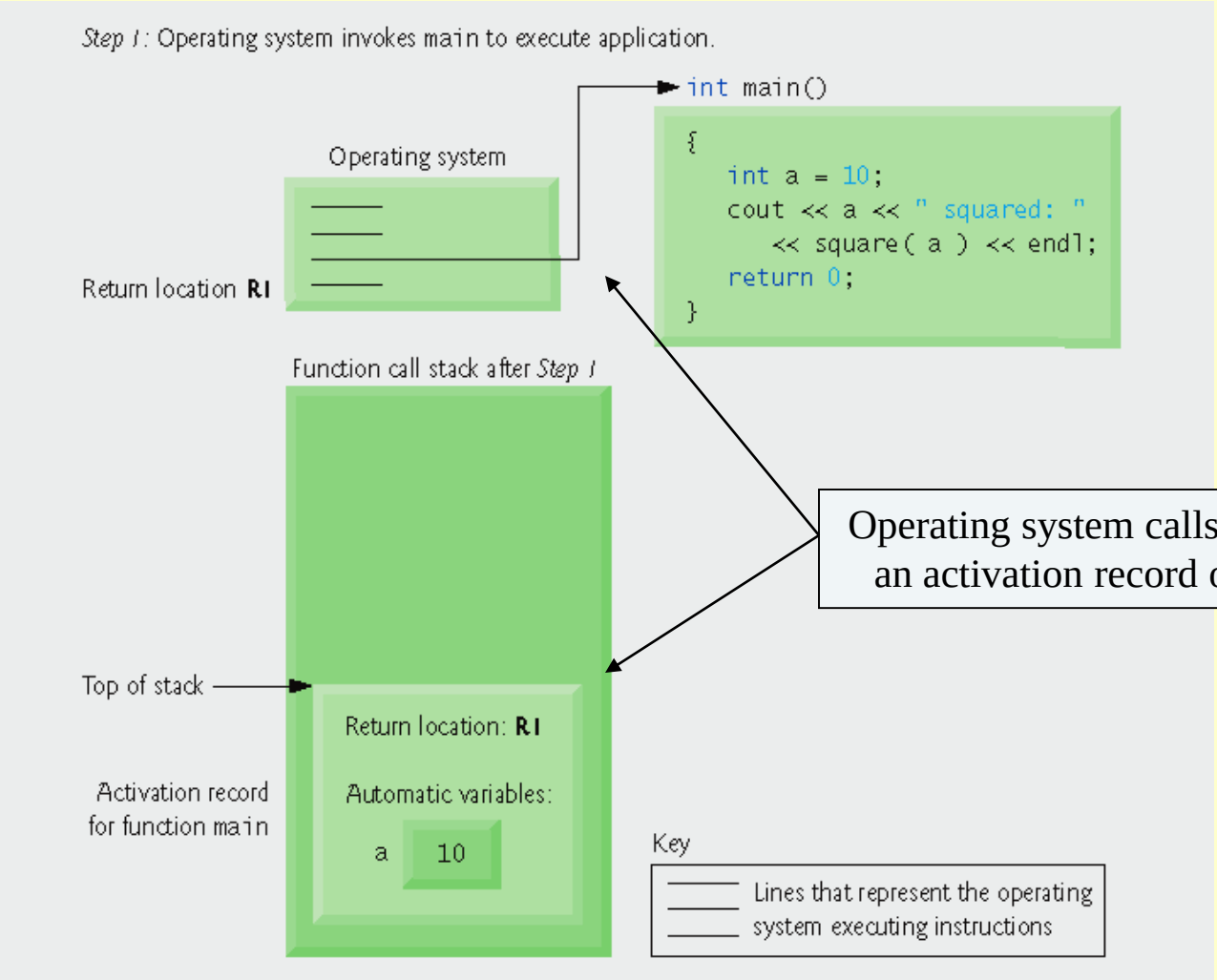


Fig. 6.14 | Function call stack after the operating system invokes `main` to execute the application.

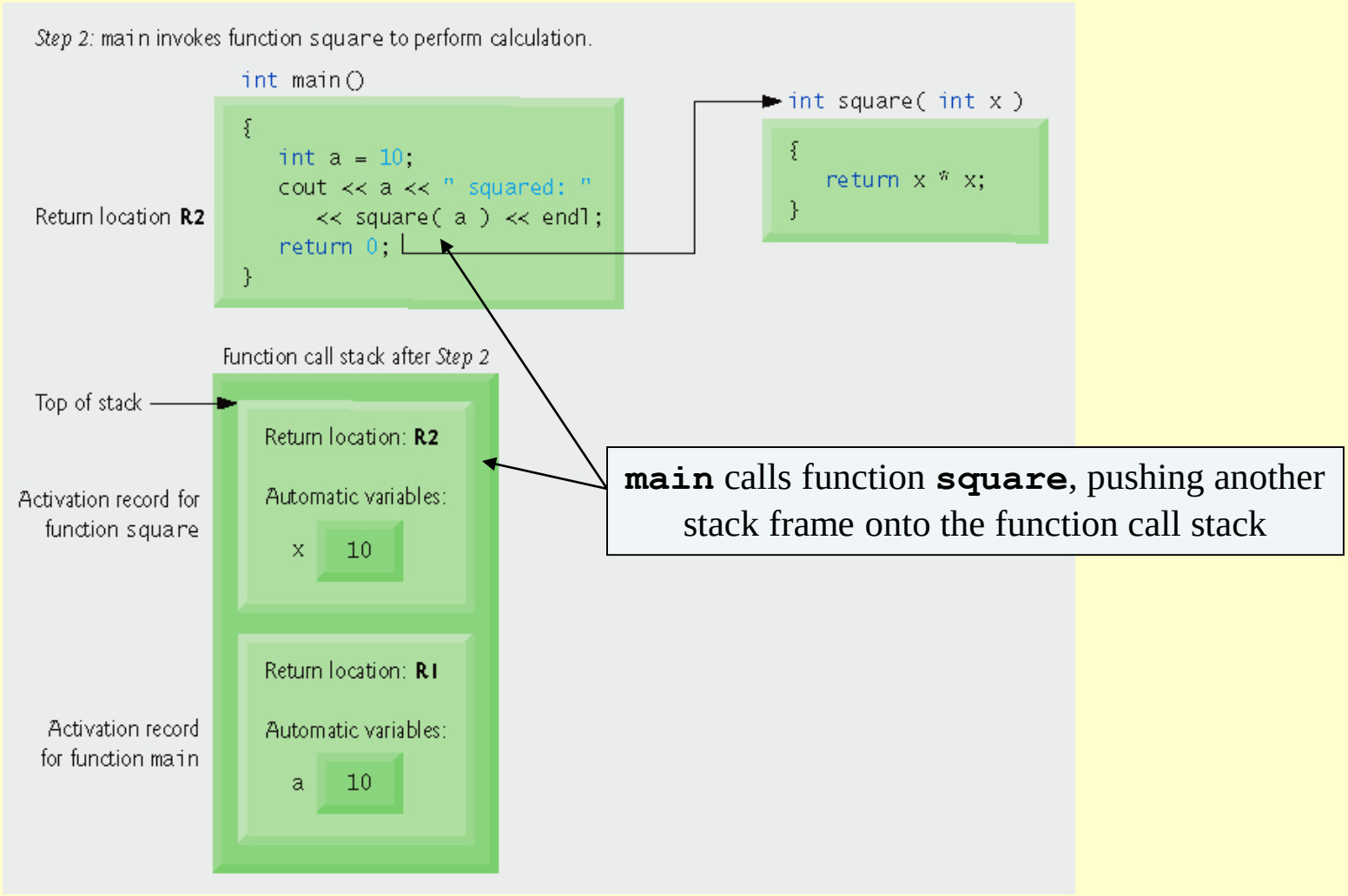


Fig. 6.15 | Function call stack after `main` invokes function `square` to perform the calculation.

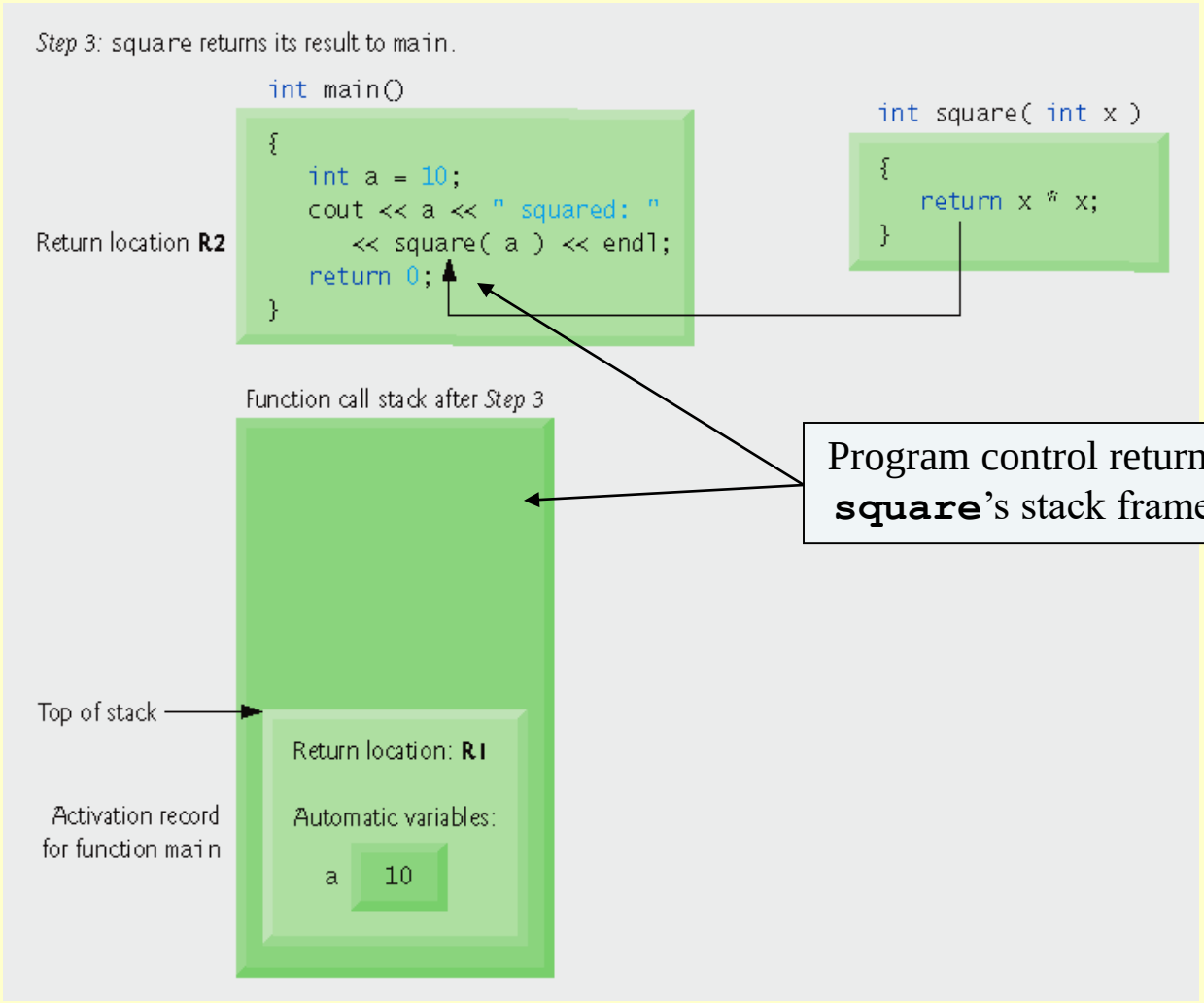


Fig. 6.16 | Function call stack after function `square` returns to `main`.

6.14 References and Reference Parameters

- **Two ways to pass arguments to functions**
  - **Pass-by-value: A *copy* of the argument's value is passed to the called function.**
    - **Changes to the copy do not affect the original variable's value in the caller.**
    - **Prevents accidental side effects of functions**
  - **Pass-by-reference: Gives called function the ability to access and modify the caller's argument data directly.**

6.14 References and Reference Parameters (Cont.)

- **Pass-by-reference**
  - **An alias for its corresponding argument in a function call**
  - **Example**
    - `int &count` **in a function header**
      - **Pronounced as "`count` is a reference to an `int`"**

```
1  // Fig. 6.19: fig06_19.cpp
2  // Comparing pass-by-value and pass-by-reference with references.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  int squareByValue( int ); // function prototype (value pass)
8  void squareByReference( int & ); // function prototype (reference pass)
9
10 int main()
11 {
12    int x = 2; // value to square using squareByValue
13    int z = 4; // value to square using squareByReference
14
15    // demonstrate squareByValue
16    cout << "x = " << x << " before squareByValue\n";
17    cout << "Value returned by squareByValue: "
18       << squareByValue( x ) << endl;
19    cout << "x = " << x << " after squareByValue\n" << endl;
20
21    // demonstrate squareByReference
22    cout << "z = " << z << " before squareByReference" << endl;
23    squareByReference( z );
24    cout << "z = " << z << " after squareByReference" << endl;
25    return 0; // indicates successful termination
26 } // end main
27
```

Outline

fig06_19.cpp

(1 of 2)

```
28 // squareByValue multiplies number by itself, stores the
29 // result in number and returns the new value of number
30 int squareByValue( int number )
31 {
32    return number *= number; // caller's argument not modified
33 } // end function squareByValue
34
35 // squareByReference multiplies numberRef by itself and stores the result
36 // in the variable to which numberRef refers in function main
37 void squareByReference( int &numberRef )
38 {
39    numberRef *= numberRef; // caller's argument modified
40 } // end function squareByReference
```

```
x = 2 before squareByValue
Value returned by squareByValue: 4
x = 2 after squareByValue

z = 4 before squareByReference
z = 16 after squareByReference
```

Outline

fig06_19.cpp

(2 of 2)

6.14 References and Reference Parameters (Cont.)

- **References**
  - **Can also be used as aliases for other variables within a function**
  - **Example:**
    - ```
      int count = 1;
      int &cRef = count;
      cRef++;
      ```
      - **Increments `count` through alias `cRef`.**
      ```

```
1  // Fig. 6.20: fig06_20.cpp
2  // References must be initialized.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  int main()
8  {
9     int x = 3;
10    int &y = x; // y refers to (is an alias for) x
11
12    cout << "x = " << x << endl << "y = " << y << endl;
13    y = 7; // actually modifies x
14    cout << "x = " << x << endl << "y = " << y << endl;
15    return 0; // indicates successful termination
16 } // end main
```

```
x = 3
y = 3
x = 7
y = 7
```

Outline

fig06_20.cpp

(1 of 1)

```
1  // Fig. 6.21: fig06_21.cpp
2  // References must be initialized.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  int main()
8  {
9     int x = 3;
10    int &y; // Error: y must be initialized
11
12    cout << "x = " << x << endl << "y = " << y << endl;
13    y = 7;
14    cout << "x = " << x << endl << "y = " << y << endl;
15    return 0; // indicates successful termination
16 } // end main
```

Borland C++ command-line compiler error message:

```
Error E2304 C:\cpphtp5_examples\ch06\Fig06_21\fig06_21.cpp 10:
   Reference variable 'y' must be initialized in function main()
```

Microsoft Visual C++ compiler error message:

```
C:\cpphtp5_examples\ch06\Fig06_21\fig06_21.cpp(10) : error C2530: 'y' :
   references must be initialized
```

GNU C++ compiler error message:

```
fig06_21.cpp:10: error: 'y' declared as a reference but not initialized
```

Outline

fig06_21.cpp

(1 of 2)

6.15 Default Arguments

- **Default argument**
  - **A default value to be passed to a parameter**
  - **Used when the function call does not specify an argument for that parameter**

Outline

```
1  // Fig. 6.22: fig06_22.cpp
2  // Using default arguments.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  // function prototype that specifies default arguments
8  int boxVolume( int length = 1, int width = 1, int height = 1 );
9
10 int main()
11 {
12    // no arguments--use default values for all dimensions
13    cout << "The default box volume is: " << boxVolume();
14
15    // specify length; default width and height
16    cout << "\n\nThe volume of a box with length 10,\n"
17       << "width 1 and height 1 is: " << boxVolume( 10 );
18
19    // specify length and width; default height
20    cout << "\n\nThe volume of a box with length 10,\n"
21       << "width 5 and height 1 is: " << boxVolume( 10, 5 );
22
23    // specify all arguments
24    cout << "\n\nThe volume of a box with length 10,\n"
25       << "width 5 and height 2 is: " << boxVolume( 10, 5, 2 )
26       << endl;
27    return 0; // indicates successful termination
28 } // end main
```

```
29
30 // function boxVolume calculates the volume of a box
31 int boxVolume( int length, int width, int height )
32 {
33    return length * width * height;
34 } // end function boxVolume
```

```
The default box volume is: 1

The volume of a box with length 10,
width 1 and height 1 is: 10

The volume of a box with length 10,
width 5 and height 1 is: 50

The volume of a box with length 10,
width 5 and height 2 is: 100
```

6.16 Unary Scope Resolution Operator

- **Unary scope resolution operator (`::`)**
  - **Used to access a global variable when a local variable of the same name is in scope.**
  - **Cannot be used to access a local variable of the same name in an outer block.**

```
1  // Fig. 6.23: fig06_23.cpp
2  // Using the unary scope resolution operator.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  int number = 7; // global variable named number
8
9  int main()
10 {
11    double number = 10.5; // local variable named number
12
13    // display values of local and global variables
14    cout << "Local double value of number = " << number
15       << "\nGlobal int value of number = " << ::number << endl;
16    return 0; // indicates successful termination
17 } // end main
```

```
Local double value of number = 10.5
Global int value of number = 7
```

Outline

fig06_23.cpp

(1 of 1)

6.17 Function Overloading

- **Overloaded functions**
  - **Overloaded functions have**
    - **Same name**
    - **Different sets of parameters**
  - **Compiler selects proper function to execute based on number, types and order of arguments in the function call.**
  - **Commonly used to create several functions of the same name that perform similar tasks, but on different data types.**

```
1  // Fig. 6.24: fig06_24.cpp
2  // Overloaded functions.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  // function square for int values
8  int square( int x )
9  {
10    cout << "square of integer " << x << " is ";
11    return x * x;
12 } // end function square with int argument
13
14 // function square for double values
15 double square( double y )
16 {
17    cout << "square of double " << y << " is ";
18    return y * y;
19 } // end function square with double argument
```

Outline

fig06_24.cpp

(1 of 2)

```
20
21 int main()
22 {
23    cout << square( 7 ); // calls int version
24    cout << endl;
25    cout << square( 7.5 ); // calls double version
26    cout << endl;
27    return 0; // indicates successful termination
28 } // end main
```

```
square of integer 7 is 49
square of double 7.5 is 56.25
```

Outline

fig06_24.cpp

(2 of 2)

6.17 Function Overloading (Cont.)

- **Overloaded functions are distinguished by their signatures.**
  - **Function signature (or simply signature): The portion of a function prototype that includes the name of the function and the types of its arguments .**
  - **Name mangling or name decoration**
    - **Compiler encodes each function identifier with the number and types of its parameters to enable type-safe linkage.**
  - **Type-safe linkage ensures that**
    - **Proper overloaded function is called.**
    - **Types of the arguments conform to types of the parameters.**

```
1  // Fig. 6.25: fig06_25.cpp
2  // Name mangling.
3
4  // function square for int values
5  int square( int x )
6  {
7     return x * x;
8  } // end function square
9
10 // function square for double values
11 double square( double y )
12 {
13    return y * y;
14 } // end function square
15
16 // function that receives arguments of types
17 // int, float, char and int &
18 void nothing1( int a, float b, char c, int &d )
19 {
20    // empty function body
21 } // end function nothing1
```

Outline

fig06_25.cpp

(1 of 2)

```
22
23 // function that receives arguments of types
24 // char, int, float & and double &
25 int nothing2( char a, int b, float &c, double &d )
26 {
27    return 0;
28 } // end function nothing2
29
30 int main()
31 {
32    return 0; // indicates successful termination
33 } // end main
```

```
@square$qi
@square$qd
@nothing1$qifcri
@nothing2$qcirfrd
_main
```

Outline

fig06_25.cpp

(2 of 2)

6.19 Recursion

- **Recursive function**
  - **A function that calls itself, either directly, or indirectly (through another function).**
- **Recursion**
  - **Base case(s)**
    - **The simplest case(s), which the function knows how to handle**
  - **For all other cases, the function typically divides the problem into two conceptual pieces.**
    - **A piece that the function knows how to do.**
    - **A piece that it does not know how to do.**
      - **Slightly simpler or smaller version of the original problem.**

6.19 Recursion (Cont.)

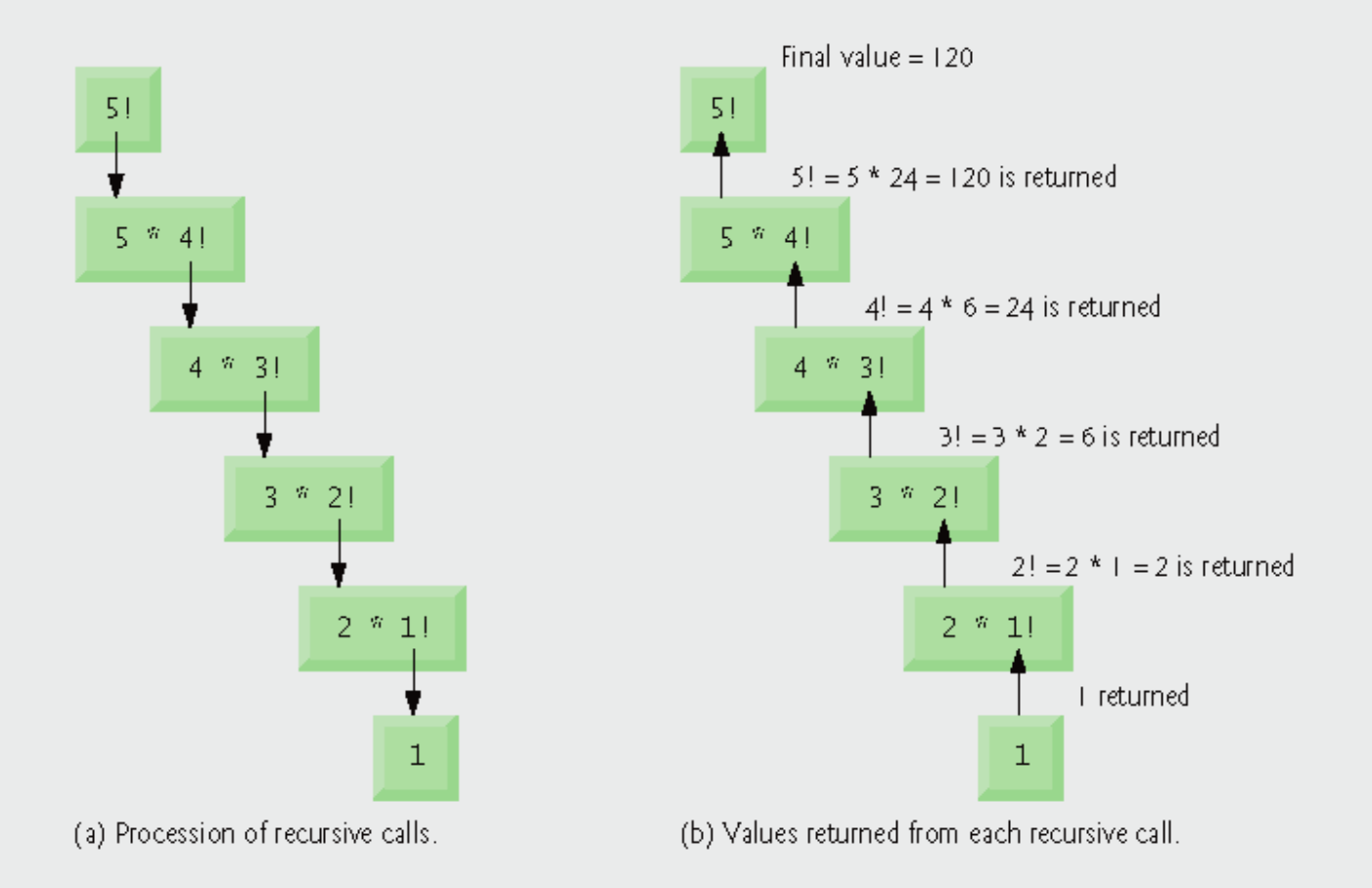
- **Recursion (Cont.)**
  - **Recursive call (also called the recursion step)**
    - **The function launches (calls) a fresh copy of itself to work on the smaller problem.**
    - **Can result in many more recursive calls, as the function keeps dividing each new problem into two conceptual pieces**
    - **This sequence of smaller and smaller problems must eventually converge on the base case.**
      - **Otherwise the recursion will continue forever.**

6.19 Recursion (Cont.)

- **Factorial**
  - **The factorial of a nonnegative integer *n*, written *n*! (and pronounced "*n* factorial"), is the product**
    - ***n* · (*n* – 1) · (*n* – 2) · … · 1**
  - **Recursive definition of the factorial function**
    - ***n*! = *n* · (*n* – 1)!**
    - **Example**
      - **5! = 5 · 4 · 3 · 2 · 1**
        5! = 5 · (4 · 3 · 2 · 1)
        5! = 5 · (4!)

6.19 Recursion (Cont.)

- **Factorial (diagram)**



```cpp
// Fig. 6.29: fig06_29.cpp
// Testing the recursive factorial function.
#include <iostream>
using std::cout;
using std::endl;

#include <iomanip>
using std::setw;

unsigned long factorial( unsigned long ); // function prototype

int main()
{
   // calculate the factorials of 0 through 10
   for ( int counter = 0; counter <= 10; counter++ )
      cout << setw( 2 ) << counter << "! = " << factorial( counter )
         << endl;

   return 0; // indicates successful termination
} // end main
```

Outline

fig06_29.cpp

(1 of 2)

```
21
22 // recursive definition of function factorial
23 unsigned long factorial( unsigned long number )
24 {
25    if ( number <= 1 ) // test for base case
26       return 1; // base cases: 0! = 1 and 1! = 1
27    else // recursion step
28       return number * factorial( number - 1 );
29 } // end function factorial
```

```
 0! = 1
 1! = 1
 2! = 2
 3! = 6
 4! = 24
 5! = 120
 6! = 720
 7! = 5040
 8! = 40320
 9! = 362880
10! = 3628800
```

Outline

fig06_29.cpp

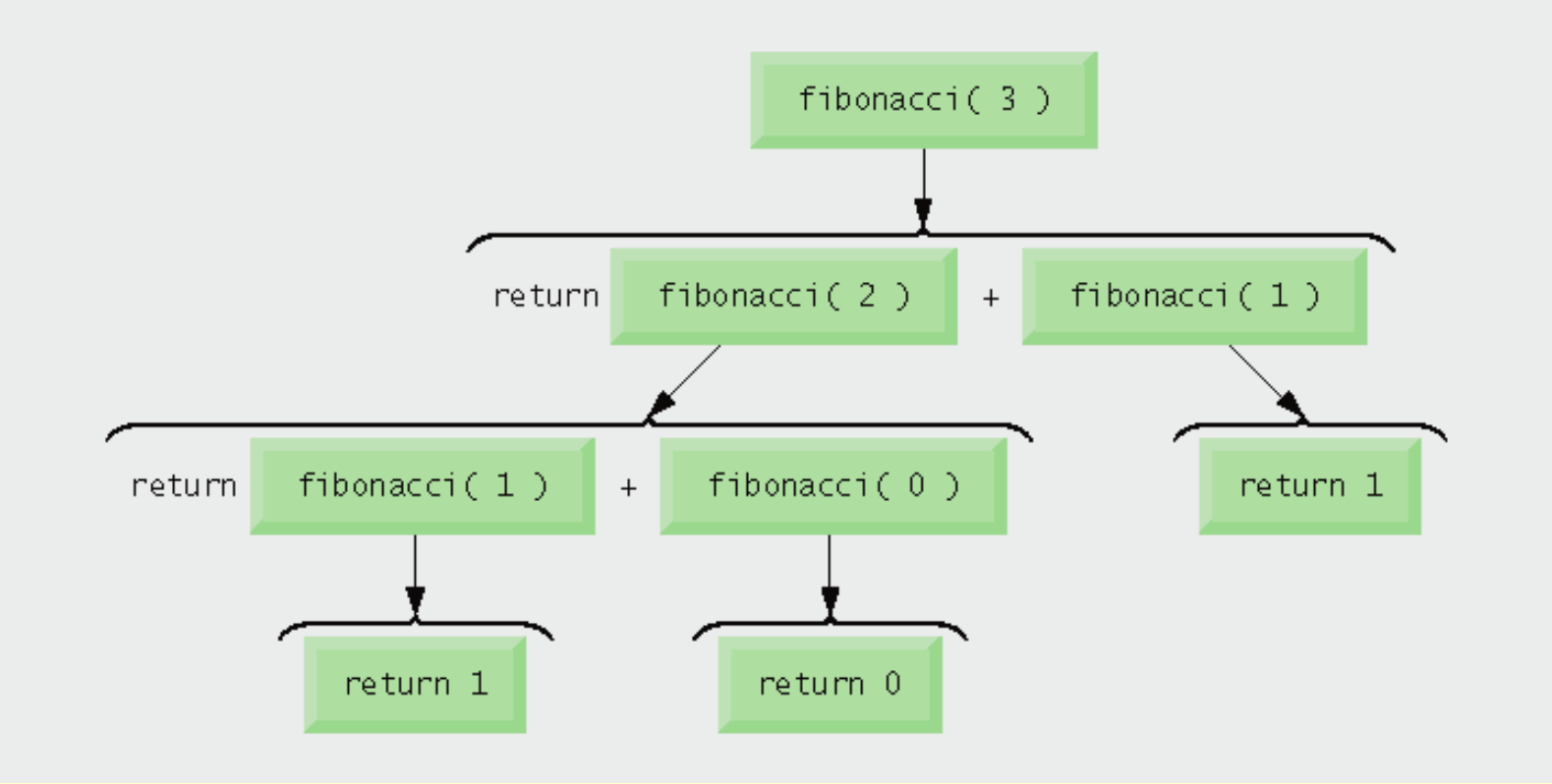
(2 of 2)

6.20 Example Using Recursion: Fibonacci Series

- **The Fibonacci series**
  - **0, 1, 1, 2, 3, 5, 8, 13, 21, …**
  - **Begins with 0 and 1**
  - **Each subsequent Fibonacci number is the sum of the previous two Fibonacci numbers**
  - **can be defined recursively as follows:**
    - **fibonacci(0) = 0**
    - **fibonacci(1) = 1**
    - **fibonacci(n) = fibonacci(n – 1) + fibonacci(n – 2)**

6.20 Example Using Recursion: Fibonacci Series

- **The Fibonacci series (diagram)**



```
1  // Fig. 6.30: fig06_30.cpp
2  // Testing the recursive fibonacci function.
3  #include <iostream>
4  using std::cout;
5  using std::cin;
6  using std::endl;
7
8  unsigned long fibonacci( unsigned long ); // function prototype
9
10 int main()
11 {
12    // calculate the fibonacci values of 0 through 10
13    for ( int counter = 0; counter <= 10; counter++ )
14       cout << "fibonacci( " << counter << " ) = "
15          << fibonacci( counter ) << endl;
16
17    // display higher fibonacci values
18    cout << "fibonacci( 20 ) = " << fibonacci( 20 ) << endl;
19    cout << "fibonacci( 30 ) = " << fibonacci( 30 ) << endl;
20    cout << "fibonacci( 35 ) = " << fibonacci( 35 ) << endl;
21    return 0; // indicates successful termination
22 } // end main
23
```

Outline

fig06_30.cpp

(1 of 2)

```
24 // recursive method fibonacci
25 unsigned long fibonacci( unsigned long number )
26 {
27    if ( ( number == 0 ) || ( number == 1 ) ) // base cases
28       return number;
29    else // recursion step
30       return fibonacci( number - 1 ) + fibonacci( number - 2 );
31 } // end function fibonacci
```

```
fibonacci( 0 ) = 0
fibonacci( 1 ) = 1
fibonacci( 2 ) = 1
fibonacci( 3 ) = 2
fibonacci( 4 ) = 3
fibonacci( 5 ) = 5
fibonacci( 6 ) = 8
fibonacci( 7 ) = 13
fibonacci( 8 ) = 21
fibonacci( 9 ) = 34
fibonacci( 10 ) = 55
fibonacci( 20 ) = 6765
fibonacci( 30 ) = 832040
fibonacci( 35 ) = 9227465
```

Outline

fig06_30.cpp

(2 of 2)

6.20 Example Using Recursion: Fibonacci Series (Cont.)

- **Caution about recursive programs**
  - **Each level of recursion in function `fibonacci` has a doubling effect on the number of function calls.**
    - `fibonacci(n)` **calls itself 2^n times.**
    - `fibonacci(20)` **requires on the order of 2^{20} or about a million calls**
    - `fibonacci(30)` **requires on the order of 2^{30} or about a billion calls.**

6.21 Recursion vs. Iteration

- **Both are based on a control statement**
  - **Iteration – repetition structure**
  - **Recursion – selection structure**
- **Both involve repetition**
  - **Iteration – explicitly uses repetition structure**
  - **Recursion – repeated function calls**
- **Both involve a termination test**
  - **Iteration – loop-termination test**
  - **Recursion – base case**

6.21 Recursion vs. Iteration (Cont.)

- **Both gradually approach termination**
  - **Iteration modifies counter until loop-termination test fails**
  - **Recursion produces progressively simpler versions of problem**
- **Both can occur infinitely**
  - **Iteration – if loop-continuation condition never fails**
  - **Recursion – if recursion step does not simplify the problem**

Outline

fig06_32.cpp

(1 of 2)

```
1  // Fig. 6.32: fig06_32.cpp
2  // Testing the iterative factorial function.
3  #include <iostream>
4  using std::cout;
5  using std::endl;
6
7  #include <iomanip>
8  using std::setw;
9
10 unsigned long factorial( unsigned long ); // function prototype
11
12 int main()
13 {
14    // calculate the factorials of 0 through 10
15    for ( int counter = 0; counter <= 10; counter++ )
16       cout << setw( 2 ) << counter << "! = " << factorial( counter )
17          << endl;
18
19    return 0;
20 } // end main
21
22 // iterative function factorial
23 unsigned long factorial( unsigned long number )
24 {
25    unsigned long result = 1;
```

```
26
27    // iterative declaration of function factorial
28    for ( unsigned long i = number; i >= 1; i-- )
29       result *= i;
30
31    return result;
32 } // end function factorial
```

```
0! = 1
1! = 1
2! = 2
3! = 6
4! = 24
5! = 120
6! = 720
7! = 5040
8! = 40320
9! = 362880
10! = 3628800
```

Outline

fig06_32.cpp

(2 of 2)

6.21 Recursion vs. Iteration (Cont.)

- **Negatives of recursion**
  - **Overhead of repeated function calls**
    - **Can be expensive in both processor time and memory space**
  - **Each recursive call causes another copy of the function (actually only the function's variables) to be created**
    - **Can consume considerable memory**
- **Iteration**
  - **Normally occurs within a function**
  - **Overhead of repeated function calls and extra memory assignment is omitted**

Until Next Time

- **Review Chapter 6**
- **Preview Chapter 7**